



FPT UNIVERSITY

Improving Small Object Detection In High-Resolution Images

Tran Van Tuan
Phi Dinh Huy
Quach Manh Duy
Nguyen Cong Tuyen

Supervisor: Assoc. Prof. Phan Duy Hung

*Bachelor of Artificial Intelligence
Hoa Lac campus - FPT University
2026*

© FPT University 2026. All rights reserved

Acknowledgment

We extend our heartfelt gratitude to Assoc. Prof. Phan Duy Hung, our dedicated and passionate supervisor, for his invaluable guidance and support in shaping our ideas over the past four months.

We also wish to thank all the lecturers at FPT University, especially those in the Information Technology Specialization Department, for equipping us with the knowledge and foundation essential for completing this thesis.

Lastly, we are deeply thankful to our families and friends for their constant encouragement and insightful feedback, which played a crucial role in helping us finalize this work.

Abstract

Small object detection remains a significant challenge in edge AI applications such as UAVs, intelligent surveillance cameras, and NVIDIA Jetson-based systems. While modern detectors like YOLO achieve high inference speed, their performance often degrades when detecting distant or tiny objects due to feature loss during downsampling. Although SAHI [9] improves detection accuracy through image slicing, applying it to entire frames introduces substantial computational overhead and limits real-time deployment on resource-constrained devices. This thesis proposes an adaptive small object detection framework that integrates YOLO, SAHI, and Reinforcement Learning (RL). The system first performs coarse detection using YOLO and extracts contextual information, including object density, confidence distribution, and small-object characteristics. Based on these states, an RL agent selectively determines where SAHI should be applied, focusing computation on informative regions while avoiding unnecessary high-resolution inference. The framework is evaluated on traffic surveillance, UAV monitoring, and security camera datasets at FullHD, 2K, and 4K resolutions. Experiments on VisDrone2019-DET-val demonstrate that RL-SAHI achieves 33.16 AP and 56.31 AP50 at 8.50 images/s, outperforming conventional SAHI and ASahi [10] in both detection accuracy and inference efficiency. In particular, RL-SAHI runs $2.30\times$ faster than SAHI and $1.62\times$ faster than ASahi while maintaining competitive detection performance. Results are expected to demonstrate that RL-guided selective slicing improves small object detection while reducing redundant computation.

Keywords: Small Object Detection, Reinforcement Learning, SAHI, YOLO, Adaptive Inference, UAV Surveillance.

Table of contents

List of tables	4
List of figures	5
1. Introduction	7
1.1. Problem & Motivation	7
1.2. Related work	8
1.2.2. Reinforcement Learning-based Methods	8
1.2.3. Slicing-based Methods	9
1.2.4. Sparse and Region-Focused Detection Methods	9
1.3. Contribution	10
2. Methodology	10
2.1. Overview Pipeline	10
2.2. Environment Setup	11
2.2.1. Data Preprocessing & Caching	11
2.2.2. The Markov Decision Process	12
2.3. Agent Design and States	13
2.3.1. State Representation	13
2.3.2. Action space and Transition dynamics	14
2.4. Reward System	14
2.4.1. Shaped environment reward	14
2.4.2. Terminal crop-outcome reward	16
2.4.3. Target-quality definition	17
2.5. RL Algorithm	18
2.6. Inference Procedure	20
3. Experiment	20
3.1. Datasets	20
3.1.1. Datasets	20
3.1.2. Dataset Characteristics and Analysis	22
3.2. Experiment settings	24
3.3. Results & Analysis	25
3.3.1. Evaluation metrics	25
3.3.2. Result Analysis	26
3.3.3. Qualitative Examples	28
4. Conclusion & Future Work	30
4.1. Conclusion	30
4.2. Future Works	31
References	33

List of tables

Table 1. Composition of the state vector	13
Table 2. Results on VisDrone2019-DET-val	27
Table 3. Component ablation on VisDrone2019-DET-val	27
Table 4. State-of-the-art comparison on VisDrone2019-DET-val	28

List of figures

Figure 1. Overview pipeline of the proposed method.	10
Figure 2. Overall architecture of the reinforcement learning framework.	18
Figure 3. Architecture of the Dueling Q-Network.	19
Figure 4. Representative validation images from the VisDrone2019-DET dataset.	21
Figure 5. Object size distribution and object density distribution of the VisDrone2019-DET dataset.	23
Figure 6. Comparison between full-image inference and slicing-based inference using SAHI.	24
Figure 7. Qualitative comparison between YOLO full-image inference and RL-SAHI on four illustrative VisDrone2019-DET images.	29

List of abbreviations and acronyms

RL:	Reinforcement learning
SAHI:	Slicing Aided Hyper Inference
RL-SAHI:	Reinforcement learning-Slicing Aided Hyper Inference
MDP:	Markov Decision Process
AP:	Average Precision
NMS:	Non-Maximum Suppression
PER:	Prioritized Experience Replay
DQN:	Deep Q-Network
GFLOPs:	Giga Floating-Point Operations
IoU:	Intersection over Union

1. Introduction

1.1. Problem & Motivation

Small object detection remains one of the critical challenges in the fields of computer vision and artificial intelligence. In numerous real-world applications—such as traffic monitoring, security surveillance, Unmanned Aerial Vehicles (UAVs), maritime observation, search and rescue operations, and smart cameras—objects frequently appear at a distance or occupy only a minimal number of pixels within a frame [17]. Consequently, conventional object detection models like YOLO struggle because crucial object features are often lost during the downsampling process, leading to low confidence scores or missed detections of small objects [18].

To enhance small object detection capabilities, various studies have adopted the Slicing Aided Hyper Inference framework [9], which divides the input image into multiple smaller patches prior to inference. This method effectively increases the relative size of objects within each patch, thereby significantly improving detection accuracy. However, applying SAHI uniformly across the entire frame drastically increases the number of inference operations, resulting in substantial computational overhead and reducing the system's processing speed.

Moreover, in practical scenarios, not every region within an image requires SAHI processing. Many frames contain only a few specific regions with small objects or high-uncertainty areas that demand detailed analysis. Applying a fixed SAHI mechanism to the entire image leads to a waste of computational resources and compromises overall system efficiency.

Beyond the challenge of computational performance, the demand for intelligent AI systems capable of context-aware self-adaptation is becoming increasingly critical. Applications such as intelligent transportation, security monitoring, UAV surveillance, urban cameras, autonomous robots, and search-and-rescue systems all demand models that achieve both high accuracy and optimized processing speeds in dynamic and complex environments. Therefore, developing an intelligent region selection mechanism, rather than a fixed full-image slicing approach, represents a highly practical and promising research direction.

Driven by these challenges, this study proposes an adaptive small object detection system based on Reinforcement Learning (RL). The system utilizes YOLO as the primary detector, integrated with SAHI as an enhancement mechanism for small object detection. An RL agent learns to automatically decide where to apply SAHI based on the state of the frame, aiming to balance detection accuracy and computational cost. This approach not only improves small object detection efficiency but also unlocks the potential for building smart, adaptive, and resource-optimized vision AI systems for a wide range of real-world applications.

1.2. Related work

1.2.1. Coarse-to-Fine Object Detection

A common approach is coarse-to-fine or adaptive zooming, in which the input image is processed at a low resolution to identify potential regions before being magnified or re-inferred at a higher resolution. Notable methods such as Dynamic Zoom-in Network [1], AutoFocus [2], AdaZoom [3], and ZoomDet [4] have demonstrated effectiveness in reducing the computational cost of processing large images while improving the detection of small objects. However, the effectiveness of these methods depends significantly on the quality of the initial region-of-interest selection stage. If small objects do not generate sufficient signals in low-resolution images, the region containing the object may not be selected for further processing, leading to the object being completely missed and having no chance of being recovered in subsequent stages.

Additionally, since processing steps are performed sequentially, errors in the early stages can propagate throughout the entire pipeline. Cropping or zooming also requires a balance between the ability to magnify the object and maintain the surrounding context, as an area that is too small may lose important contextual information, while inappropriate scaling transformations can affect the object's shape and the accuracy of the bounding box.

1.2.2. Reinforcement Learning-based Methods

Reinforcement learning (RL) is also used to coordinate high-resolution image processing. Studies such as Active Object Localization [5], Dynamic Zoom-in Network [1], Efficient Object Detection in Large Images using Deep Reinforcement Learning [6], Scale Optimization Using Evolutionary Reinforcement Learning [7], and AIRS [8] demonstrate that agents can learn to select observation regions, adjust resolution, or optimize scale to reduce computational costs. The advantage of this group of methods is their ability to make adaptive decisions based on image content rather than applying a fixed strategy to the entire dataset. However, their effectiveness depends significantly on the quality of the region selection or scale selection policy. Suboptimal decisions in the early stages may cause some small objects to be processed at an inappropriate resolution, leading to object omissions. Furthermore, the sequential decision-making process often introduces additional computational overhead, while the design of the reward function and action space must be carefully balanced to ensure training stability. Therefore, although RL is a promising approach for the computational resource allocation problem, maintaining detection accuracy while reducing processing costs remains a significant challenge.

1.2.3. Slicing-based Methods

Currently, slicing-based inference methods such as SAHI [9] are widely used for the problem of detecting small objects in high-resolution images. SAHI divides the image into many small patches, performs inference on each patch, and then merges the results into the original image coordinate system. This approach helps increase the relative size of small objects in the model’s input, while also allowing direct integration with many existing detectors without requiring architectural changes or model retraining. However, traditional slicing strategies typically use a fixed grid or slice size, leading to the processing of regions containing only background and increasing the number of forward passes as well as inference time.

ASahi [10] has reduced some of this redundant computation by adjusting the number and size of slices based on image resolution. Nevertheless, this strategy still relies primarily on general rules related to image size and does not directly leverage information about content distribution within the image to adjust the slicing process in a more adaptive manner.

1.2.4. Sparse and Region-Focused Detection Methods

Sparse or Region-focused methods such as ClusDet [11], QueryDet [12], ESOD [13], AD-Det [14], and Dynamic Adaptive Region Transformer [15] also aim to reduce computational overhead by focusing on regions likely to contain small objects. These methods leverage the non-uniform distribution of objects, sparse query mechanisms, or key region detection modules to avoid uniform processing of the entire image. However, their effectiveness often depends on region selection mechanisms or specialized computations deeply integrated into the detector architecture. If key regions are not accurately identified, some small objects may not receive the necessary level of processing. Furthermore, applying these methods often requires modifying the detector’s architecture or training process, reducing flexibility when deploying it as an independent inference component.

Research gap. Although previous studies have demonstrated several effective approaches to improving the detection of small objects in high-resolution images, there remains a trade-off between accuracy, speed, and flexibility. Zoom-in methods depend on the quality of the initial region of interest; RL methods require control over overhead, rewards, and the action space; slicing-based inference is easy to implement but suffers from redundancy due to a fixed grid; and sparse or region-focused methods often rely on specialized modules. Therefore, the thesis **Improving Small Object Detection in High-Resolution Images** aims to select slice regions based on image content rather than scanning the entire image with a fixed grid, instead of applying a fixed image-splitting strategy across the entire image space. This approach aims to concentrate computational resources on regions likely to contribute more to small

object detection, while controlling the number and size of slices to balance accuracy and inference cost.

1.3. Contribution

The main contribution of this thesis is the development of an **adaptive slicing mechanism** based on Reinforcement Learning (RL) for small object detection in high-resolution images. Instead of exhaustively applying SAHI over a fixed grid, the proposed approach learns to determine where high-resolution slicing should be performed according to image content. This strategy aims to reduce redundant slices while preserving the detection performance of small objects. Furthermore, the proposed method maintains the detector-agnostic at the slicing and merging level advantage of slicing-based inference and can be integrated with existing object detectors without requiring modifications to their architectures. By selectively allocating computational resources to informative image regions, the method seeks to achieve a better trade-off between detection accuracy and computational cost.

2. Methodology

2.1. Overview Pipeline

The proposed coarse-to-fine design follows the general principle of selectively applying high-resolution detection to informative regions rather than uniformly processing all image areas [1, 2, 6].

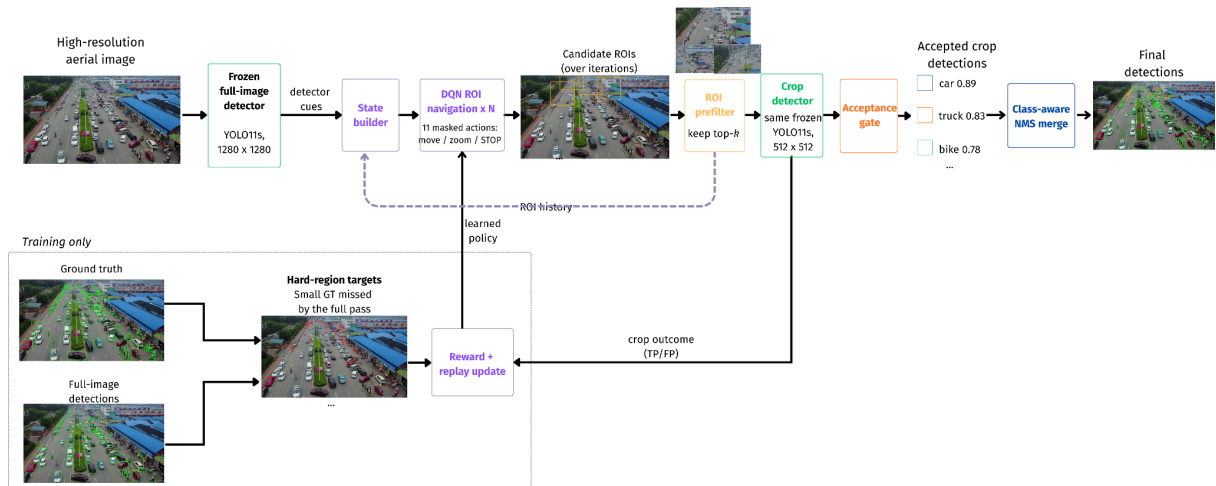


Figure 1. Overview pipeline of the proposed method.

Figure 1 shows the complete pipeline. A single pass of the frozen detector over the full image produces both the base detections that enter the final merge and the cues from which the state builder assembles the observation. The policy then navigates a region of interest over N iterations using eleven masked actions - eight translations, two scale changes and STOP - each iteration contributing one candidate region; the history of proposed regions is fed back into the state so that the same area is not revisited. The ROI pre-filter ranks the candidates on the cached cues alone and keeps

the top k , which are cropped and passed through the same frozen detector. Crops that survive the acceptance gate are merged with the base detections by a single class-aware non-maximum suppression pass, following the slice-infer-merge paradigm of SAHI [9]. The dashed block is used only during training: hard-region targets are mined by comparing the ground truth with the full-image detections and, together with the true- and false-positive outcome of each crop, form the reward that updates the policy through replay.

2.2. Environment Setup

2.2.1. Data Preprocessing & Caching

The detector was trained and evaluated on high-resolution drone-captured aerial images with YOLO-format annotations. The dataset follows VisDrone [19] naming and labeling conventions and is split into standard training, validation, and test partitions. All ten original VisDrone2019-DET categories are used throughout the pipeline: pedestrian, people, bicycle, car, van, truck, tricycle, awning-tricycle, bus, and motor. A class-mapping module harmonizes the label spaces used by the detector, the dataset annotations, and the evaluation protocol, ensuring that training, reward computation, and evaluation operate on the same ten-class set. Before training, the pipeline verifies that the class sets used for hard-region mining and inference are identical; otherwise, training is aborted to prevent silent label drift.

For each partition, the effective dataset size is defined as the number of images with valid and up-to-date cache entries at loading time. Because the RL policy observes detector-derived states rather than raw pixels, the representativeness of the learned policy depends primarily on the diversity of scenes, flight altitudes, object densities, and class distributions in the source partition.

To make RL training computationally tractable, expensive perception steps are pre-computed and cached. The agent does not re-run the detector when constructing state observations. Three cache artifacts are generated:

Full-image detection. YOLO11s [16] is run once per image with input size 1280, confidence threshold 0.01, IoU threshold 0.7, and a maximum of 3000 detections. The low confidence threshold is used to retain weak proposals. The resulting DetectionCache stores bounding boxes, confidence scores, predicted classes, a 256-dimensional semantic feature vector computed from the spatial mean and standard deviation of backbone layer 16, a 16×16 objectness-like heatmap derived from the maximum class logit, and four compressed spatial feature maps extracted from YOLO layer 6.

Hard-region mining. During training, ground-truth annotations are used to identify small objects with relative area less than 32 px that are missed by the full-image detector. These regions are stored in a HardRegionCache.

Crop-outcome cache. During training, detections generated from evaluated crops are cached on disk. Each entry is keyed by a hash of the ROI, detector weights, and detector parameters, allowing repeated episodes to reuse previously computed crop outcomes.

Each cache stores a hash value of the detector configuration. An image is only accepted into the RL training process if it has valid and up-to-date caches for detection and hard regions. Missing or outdated cache entries are ignored and reported along with the reason, making data quality issues easier to identify. During inference, a mismatch between the checkpoint metadata and the active configuration will result in an error rather than producing potentially inconsistent predictions.

All observations stored in the cache are normalized before being used by the policy. Feature vectors are L2-normalized and clipped to $[-5, 5]$. Spatial feature maps are normalized by the z-score and clipped. Object maps are passed through a sigmoid function, real-valued summary features are normalized using configured constants, and all maps are resized using bilinear interpolation to a common 16×16 grid..

2.2.2. The Markov Decision Process

The selection of regions of interest (ROIs) is modeled as a Markov Decision Process (MDP) [28], defined by the quadruple (S, A, P, R, γ) , with a discount factor $\gamma = 0.95$. This formulation follows the active object localization paradigm of Caicedo and Lazebnik [5], in which an agent transforms a bounding box through a sequence of discrete actions until it emits a terminal action. In this model, each episode corresponds to a single ROI rollout. A complete image is processed through a sequence of ROI iterations: the agent proposes and refines an ROI, after which that ROI is either accepted or rejected, and a new iteration begins if the image-level slice budget has not been exhausted.

Two ROI histories are maintained throughout the processing of each image. The accepted ROI history contains the slices used for the final detection fusion and the class-based non-maximum suppression (NMS) method, and also contributes to the overlap penalties. The tested ROI history contains all proposed ROIs, including both accepted and rejected ones, and is used to limit the repeated exploration of similar regions. Each ROI iteration is limited to 5 agent steps. For each image, the policy can accept a maximum of 4 slices out of a maximum of 4 slice attempts.

2.3. Agent Design and States

2.3.1. State Representation

At each decision step, the agent observes a state representation constructed from the features of the detector, the spatial map, the region of interest (ROI) history, and numerical summary statistics. The observation data is stored as a concatenated vector, but its components retain both vector-based and grid-based information. G is the

spatial grid size, with $G = 16$ in the default configuration. The total dimension of the state is:

$$D = F + C_{map} \times G \times G + 28 \quad (1)$$

where F is the dimension of the detector feature vector and $C_{map} = 13$ is the number of spatial map channels. In the default settings, this results in a state vector with 3612 dimensions, depending on the dimension of the detector features stored in the buffer, listed in **Table 1**.

Table 1. Composition of the state vector.

Component	Shape	Description
Feature vector	256	L2-normalized semantic feature vector extracted from YOLO backbone layer 16
History map	$1 \times 16 \times 16$	Grid cells visited by the current ROI during the current rollout
Current-ROI map	$1 \times 16 \times 16$	Rasterized current ROI at the current decision step
Attempted-slice map	$1 \times 16 \times 16$	Rasterized ROIs that have already been tested
Accepted-slice map	$1 \times 16 \times 16$	Rasterized ROIs accepted for final detection fusion
Detector-cue maps	$4 \times 16 \times 16$	High-confidence detections, proposal quality, proposal density, and small-object channels
Objectness map	$1 \times 16 \times 16$	Objectness-like heatmap derived from the detector output
Spatial feature maps	$4 \times 16 \times 16$	Four compressed spatial feature maps extracted from YOLO layer 6
Summary vector	28	Scalar statistics describing detections, ROI geometry, progress, overlap, and slice usage

A 28-dimensional summary vector encodes both image-level and region-of-interest (ROI)-level detection statistics, including the number of detections, average and maximum confidence, as well as the percentage of low-confidence, small-sized detections, and detection proposals. It also includes the center, width, height, and area of the normalized ROI; the normalized step progression; the history range; overlap with previously accepted slices; the scaling factor $\eta = \min(H, W) / \text{side (ROI)}$; and the number of slices accepted to date. The use of detector evidence and action history is inspired by prior sequential region-selection approaches [1, 3].

The spatial component contains 13 channels: four ROI-history channels, four detector-cue channels, one objectness channel, and four compressed spatial-feature channels. The state layout is inferred from the detector outputs stored in the buffer, allowing the network input size to remain consistent with the actual feature size stored in the buffer.

2.3.2. Action space and Transition dynamics

Following the discrete transformation set introduced for active object localization [5], the agent has eleven actions: four axis-aligned translations (LEFT, RIGHT, UP, DOWN), four diagonal translations (UP-LEFT, UP-RIGHT, DOWN-LEFT, DOWN-RIGHT), two scale changes (ZOOM-IN, ZOOM-OUT), and a terminal STOP. A translation shifts the ROI by $\delta = 0.65$ side (ROI), with diagonal moves scaled by $\frac{1}{\sqrt{2}}$ to preserve isotropic speed. A scale change multiplies the ROI side by 0.75 for zoom-in or by its reciprocal for zoom-out, clamped to between 0.10 and 0.35 of the shorter image side and to a maximum area ratio of 0.12. The stop action terminates the rollout; hard-region coverage is committed only when stop is selected. The transition function is deterministic: the next ROI is obtained by applying the chosen action and clipping it to the image bounds.

A valid-action mask constrains both exploration and the learning target. Actions that would not change the ROI, such as moving against an image border or applying a saturated zoom, are masked. If a non-stop action exists that can escape an overlapping region, moves that keep the ROI above the overlap threshold are masked. The stop action is disabled when the current ROI still overlaps a previously accepted or attempted region above the threshold while an escape move remains available. The initial ROI of each rollout is evidence-seeded rather than fixed at the image center: it is placed at the peak of an objectness-based heatmap, falling back to the highest-priority proposal or small uncertain detection, while down-weighting previously attempted regions.

2.4. Reward System

2.4.1. Shaped environment reward

The reward function was designed to encourage the agent to select compact regions that improve the detectability of hard small objects, while discouraging redundant exploration and unnecessary crop evaluation. Let α_r denote the area ratio of the current ROI relative to the full image, and let η denote its scale gain. The variables o_{old} and o_{att} represent the overlap ratios between the current ROI and the previously accepted and attempted ROIs, respectively. In addition, o_{det} denotes the normalized

overlap between the ROI and high-confidence detections obtained from the full-image detector.

Let H be the set of hard regions. For a given ROI, n_{new} denotes the number of newly captured hard regions, and q_{new} denotes their aggregated target-quality score. The term n_{cand} represents the number of candidate hard regions currently covered by the ROI, whereas n_{ret} denotes the number of previously covered hard regions revisited by the current ROI. To provide intermediate guidance during navigation, the hard-region coverage progress is defined as $\Delta n_{cand} = n_{cand} - n_{cand}^{prev}$. Let $[x]_+ = \max(x, 0)$.

The hard-region reward combines terminal target acquisition, intermediate coverage progress, and penalties for empty or repeated regions:

$$\begin{aligned} R_{hard} = & 1[H \neq \emptyset][1[a = STOP](0.75n_{new} + 0.225 \cdot \text{clip}(\frac{q_{new}p_{max}}{a_r}, 0.3))] \\ & + 1[a \neq STOP](0.5 \cdot \text{clip}(\Delta n_{cand}, -4, 4)) \\ & - c_a 1[n_{cand} = 0] - 0.5 \min(n_{ret}, 4) \end{aligned} \quad (2)$$

where $p_{max} = 0.12$ is the maximum allowable ROI area ratio, $c_a = 1$ for the STOP action and $c_a = 0.25$ for non-terminal actions. Accordingly, the agent receives a positive reward only when a STOP action commits newly captured hard regions. During non-terminal transitions, the reward provides dense feedback according to the improvement in hard-region coverage. Conversely, regions containing no candidate hard object or repeatedly covering previously accepted hard regions are penalized.

To account for computational efficiency, each transition receives an area-dependent penalty:

$$R_{eff} = -0.5(0.05 + 0.5a_r) \quad (3)$$

This term favours smaller ROIs and discourages the agent from relying on unnecessarily large crops, which would reduce the computational advantage of adaptive slicing.

The constraint reward enforces the desired geometric properties of a useful crop. It penalizes ROIs that are excessively large, insufficiently zoomed, or highly overlapping with regions that have already been accepted or attempted:

$$\begin{aligned} R_{con} = & -3.0 \left(\left[\frac{a_r}{p_{max}} - 1 \right]_+ + \left[\frac{\eta_{min}}{\eta} - 1 \right]_+ + 1[o_{old} \geq 0.5] + \frac{2}{3} 1[o_{att} \geq 0.5] \right. \\ & \left. - o_{det} \right) \end{aligned} \quad (4)$$

where $\eta_{min} = 2.5$. The first two terms promote compact ROIs with sufficient enlargement, whereas the overlap penalties prevent repeated inspection of previously explored regions. The final term further discourages the selection of regions that are already well explained by high-confidence full-image detections.

A terminal STOP action is additionally evaluated according to the quality of the selected region. When the ROI captures newly uncovered hard regions, the bonus is proportional to the corresponding target quality. If hard-region supervision is unavailable, an observable target score z , derived from detector evidence, is used instead:

$$R_{stop} = \begin{cases} 0.5\min(q_{new}, 4), & n_{new} > 0 \wedge o_{old} < 0.5, \\ 0.15\min(z, 2), & H = \emptyset \wedge z > 0.3, \\ -0.25 & \text{otherwise} \end{cases} \quad (5)$$

The final per-step reward is computed as $r_t = R_{hard} + R_{con} + R_{eff} + R_{STOP}$. In addition, stalled non-terminal actions and rollouts that reach the step limit without an explicit STOP action are forced to receive a non-positive reward and an additional penalty. Similar penalties are applied to ROIs that overlap excessively with accepted or attempted regions. These terms are not potential-based reward shaping; rather, they are introduced as a practical mechanism for stabilizing learning and promoting efficient, non-redundant ROI selection.

2.4.2. Terminal crop-outcome reward

The shaped environment reward provides guidance based on hard-region geometry and detector evidence. However, a geometrically plausible ROI is useful only if running the detector on the corresponding crop produces a meaningful contribution to the final detection set. Therefore, for each terminal proposal eligible for crop evaluation, the system crops the selected ROI, performs detector inference, maps the predicted boxes back to the original-image coordinate system, and evaluates the crop relative to the full-image detections and all previously accepted crop detections.

A crop detection is considered novel if it survives the configured class-aware merging procedure and does not duplicate a pre-existing detection of the same class at the duplicate threshold of 0.5 IoU. Let Δ_{new} denote the number of novel detections contributed by the crop, U_{new} denote the sum of their confidence scores, and s_{max} denote the maximum confidence score among them. When ground-truth annotations are available, the true-positive and false-positive gains, Δ_{TP} and Δ_{FP} , are obtained by comparing the merged detections before and after adding the crop. Matching is

performed greedily on a one-to-one basis at IoU 0.5. When annotations are unavailable, both Δ_{TP} and Δ_{FP} are set to zero.

Let m denote the number of valid detections produced by the crop after class filtering. The crop-outcome reward is defined as

$$R_{crop} = 0.5 \cdot U_{new} + 3.0 \cdot [\Delta_{TP}]_+ - 0.5 \cdot [\Delta_{FP}]_+ - 1.21[m = 0] - 1.21[m > 0 \wedge \Delta_{new} = 0] \quad (6)$$

where $[x]_+ = \max(x, 0)$. The first term rewards crops that contribute high-confidence novel detections. The second and third terms encourage improvements in true positives while penalizing increases in false positives. The last two terms penalize empty crops and crops that produce detections but do not contribute any novel result after merging. These two penalties are mutually exclusive.

A crop is accepted when it contributes at least one novel detection, the maximum novel-detection confidence is at least 0.1, and either the crop contributes exactly one novel detection or its total novel-detection utility is at least 0.3. Formally, the acceptance criterion is $\Delta_{new} \geq 1 \wedge s_{max} \geq 0.1 \wedge (\Delta_{new} = 1 \vee U_{new} \geq 0.3)$. This condition allows a confident isolated detection to be retained while requiring stronger aggregate evidence when a crop contributes multiple detections.

The terminal reward combines the shaped environment reward with the crop-outcome reward:

$$r_T = \begin{cases} r_{base} + 0.5n_{new} + R_{crop} - 0.25 & \text{if the slice is accepted} \\ \min(r_T^{base}, 0) + \min(R_{crop}, 0) - 0.5 - 0.25 & \text{if the slice is rejected} \end{cases} \quad (7)$$

Here, r_{base} is the shaped reward defined in Section 2.4.1, and n_{new} is the number of newly committed hard regions. The accepted case adds the crop outcome and the hard-region contribution, while the rejected case prevents a positive shaped reward from compensating for an ineffective crop. The fixed deductions also account for the cost of evaluating a crop proposal and discourage repeated unsuccessful attempts.

2.4.3. Target-quality definition

A hard region is counted as well captured only if the ROI is sufficiently tight and zoomed ($\alpha_r \leq p_{max}$ and $\eta \geq \eta_{min}$), the object's centre lies inside the ROI with margin, and the object's projected size, once the crop is resized to the reward resolution of 320 pixels, falls within a detector-friendly band between 12 and 96 pixels centred on 32 pixels - the COCO threshold below which an object is classified as small [29]. This

formalises the notion that a useful slice makes a small object large enough to be detected.

2.5. RL Algorithm

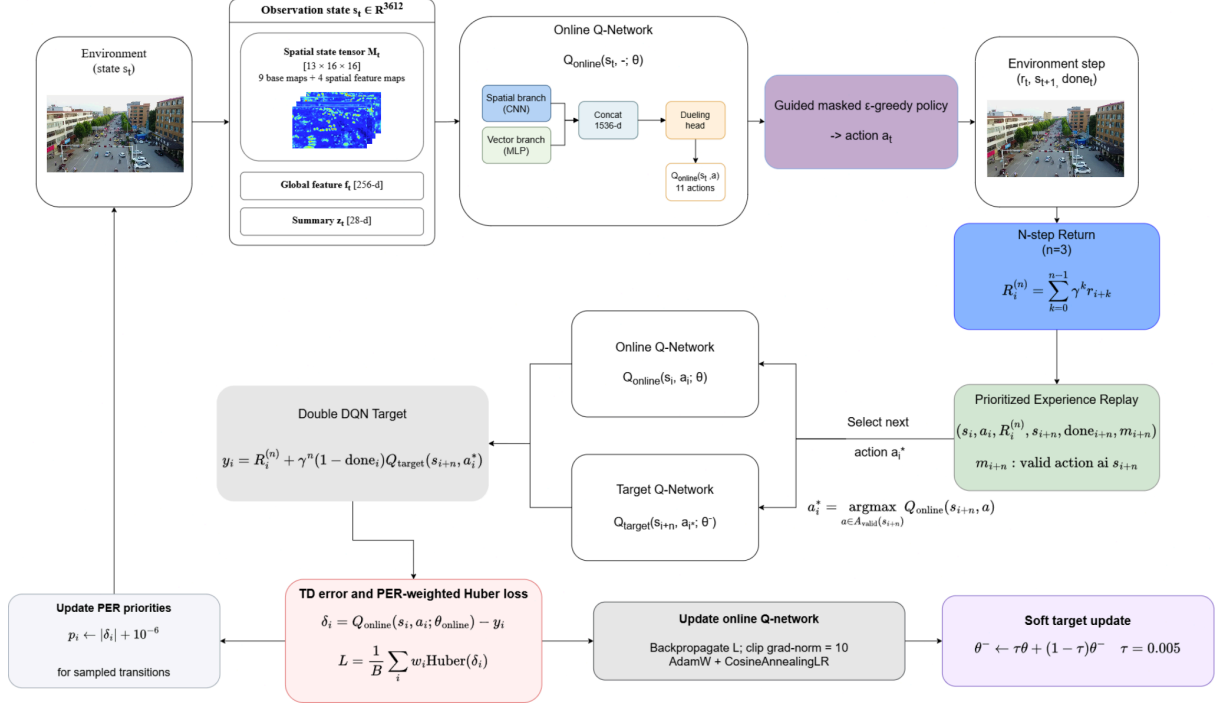


Figure 2. Overall architecture of the reinforcement learning framework.

The learner is a Dueling Double DQN [21, 22] as shown in **Figure 2**, built on the deep Q-network [20] and augmented with Prioritized Experience Replay, N-step returns, and soft target updates, optimised with AdamW and a cosine-annealed learning rate. Replay stores N-step transitions $(s_t, a_t, R_t^{(n)}, s_{t+n}, d_{t+n}, m_{t+n})$, where d_{t+n} is the terminal flag, m_{t+n} the valid-action mask at s_{t+n} , and

$$R_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k}, \quad n = 3 \quad (8)$$

The masked Double-DQN target uses the online network to select the next action among valid actions and the target network to evaluate it:

$$a_t^* = \underset{a: m_t[a]=1}{\operatorname{argmax}} Q_{\theta}(s_t, a), y = R_t^{(n)} + \gamma^n (1 - d_t) Q_{\theta^-}(s_t, a_t^*) \quad (9)$$

Masking invalid actions before the argmax prevents value over-estimation arising from infeasible moves (border-blocked translations or saturated zooms). With the temporal-difference error $\delta = Q_{\theta}(s_0, a_0) - y$, the loss is the importance-weighted Huber loss of prioritized replay [23]:

$$L(\theta) = E_j[w_j \cdot \text{Huber}(\delta_j)], w_j = \left(\frac{1}{N \cdot P(j)}\right)^\beta / \max_k w_k, p_j \propto (|\delta_j| + \varepsilon)^\alpha \quad (10)$$

with priority exponent $\alpha = 0.6$ and importance exponent β annealed from 0.4 to 1.0 over 100,000 prioritized replay sampling updates; the gradient norm is clipped to 10. The target network is updated softly after each optimisation step:

$$\theta^- \leftarrow \tau \theta + (1 - \tau) \theta^-, \tau = 0.005 \quad (11)$$

which is more stable than a periodic hard copy. Exploration proceeds from exploration to exploitation through three mechanisms acting together: an ε -greedy rule over valid actions, with ε decaying linearly from 1.0 to 0.05 over 15,000 steps; a guided-action demonstrator invoked with probability decaying from 0.25 to 0.05, which escapes overlap or moves toward the best un-tried objectness or proposal peak and knows when to zoom or stop, used to bootstrap learning while the network is still untrained; and a curriculum in which the per-image slice budget grows from one to four over 15,000 steps, so the agent first masters single-object cropping before multi-slice coordination.

The action-value function is approximated by a two-stream network with a dueling output, as shown as **Figure 3**.

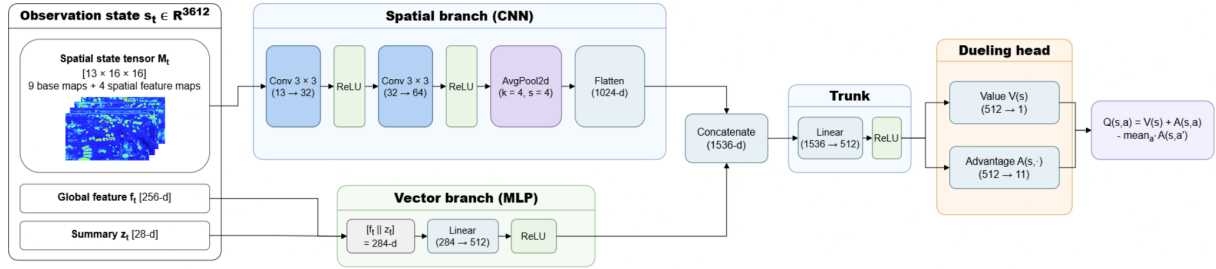


Figure 3. Architecture of the Dueling Q-Network.

The 3,612-dimensional observation is split into a $13 \times 16 \times 16$ spatial tensor and a 284-dimensional vector. The spatial tensor contains four ROI-history maps, four detector-cue maps, one objectness map, and four compressed spatial maps from YOLO layer 6. The spatial branch applies two 3×3 convolutional layers ($13 \rightarrow 32 \rightarrow 64$), followed by a fixed 4×4 average-pooling operation and flattening to 1,024 dimensions. The vector branch concatenates the 256-dimensional detector feature vector and the 28-dimensional summary vector, then projects the resulting 284-dimensional vector to 512 dimensions. The two branches are concatenated into a 1,536-dimensional vector and reduced to 512 dimensions by the shared trunk, which feeds a dueling head: one linear layer produces the scalar state value $V(s)$, another the advantage $A(s, \cdot)$ over the eleven actions. The two are recombined as

$$Q(s, a) = V(s) + A(s, a) - \text{mean}_a A(s, a') \quad (12)$$

which is the dueling aggregation of Wang et al. [22]; it separates the value of a state from the effect of the action and accelerates learning where the choice of action matters little. The greedy policy takes the argmax over valid actions only.

2.6. Inference Procedure

At inference the frozen YOLO11s is applied once to the full image; the detections, the layer-16 feature vector, the objectness map and the compressed spatial maps it produces form the static state reused by every rollout. The policy then performs greedy rollouts, each proposing one region and terminating on STOP, on a stalled region or at the step limit, with earlier regions fed back through the attempted-slice channel and the overlap terms of the action mask so that the same area is not proposed twice; no detector call is made during this stage. A pre-filter ranks the proposals using only the cached full-image evidence and keeps the k highest-scoring regions, which are cropped and evaluated as a single batch, so an image costs one full-image pass plus at most k crops. A crop is retained only if it contributes novel detections under the rule of Equation (7), and the full-image and retained-crop detections are combined by a single class-aware non-maximum suppression pass. The settings are given in Section 3.2.

3. Experiment

3.1. Datasets

3.1.1. Datasets

In this study, the VisDrone2019-DET dataset was used as the primary dataset for training, validation, and evaluation. VisDrone2019-DET is a widely used benchmark dataset for aerial object detection, consisting of images captured by unmanned aerial vehicles (UAVs) in various real-world urban environments. The dataset contains scenes from roads, residential areas, traffic intersections, parking lots, and other public urban spaces. These scenes provide realistic conditions for evaluating object detection models under complex aerial surveillance scenarios.

The selection of VisDrone2019-DET is closely aligned with the objective of this research, which is to improve small object detection in high-resolution images. In UAV-based images, objects often occupy only a small portion of the entire image due to the wide field of view and the long distance between the camera and the objects. When these high-resolution images are resized to a fixed input size for object detection models, small objects may lose important visual details and become more difficult to detect. Therefore, a dataset with a high proportion of small objects is necessary to properly evaluate the limitations of conventional full-image inference and the effectiveness of slicing-based inference methods.

VisDrone2019-DET contains 10 object categories, including pedestrian, people, bicycle, car, van, truck, tricycle, awning-tricycle, bus, and motor. These classes

represent common objects in urban traffic and aerial monitoring scenarios. The dataset includes both human-related and vehicle-related categories, allowing the detection model to be evaluated across objects with different shapes, scales, and visual characteristics. In this research, all 10 original classes were retained instead of filtering or merging categories. This decision helps preserve the original complexity of the dataset and provides a more realistic multi-class object detection setting.

The original annotation format of VisDrone was converted into YOLO format to match the training and inference pipeline used in this study. In YOLO format, each object annotation is represented by the class index, the normalized center coordinates of the bounding box, and the normalized width and height of the bounding box. This conversion ensures that the dataset is compatible with the YOLO-based detection framework and that all annotations follow a consistent format during model training and evaluation.

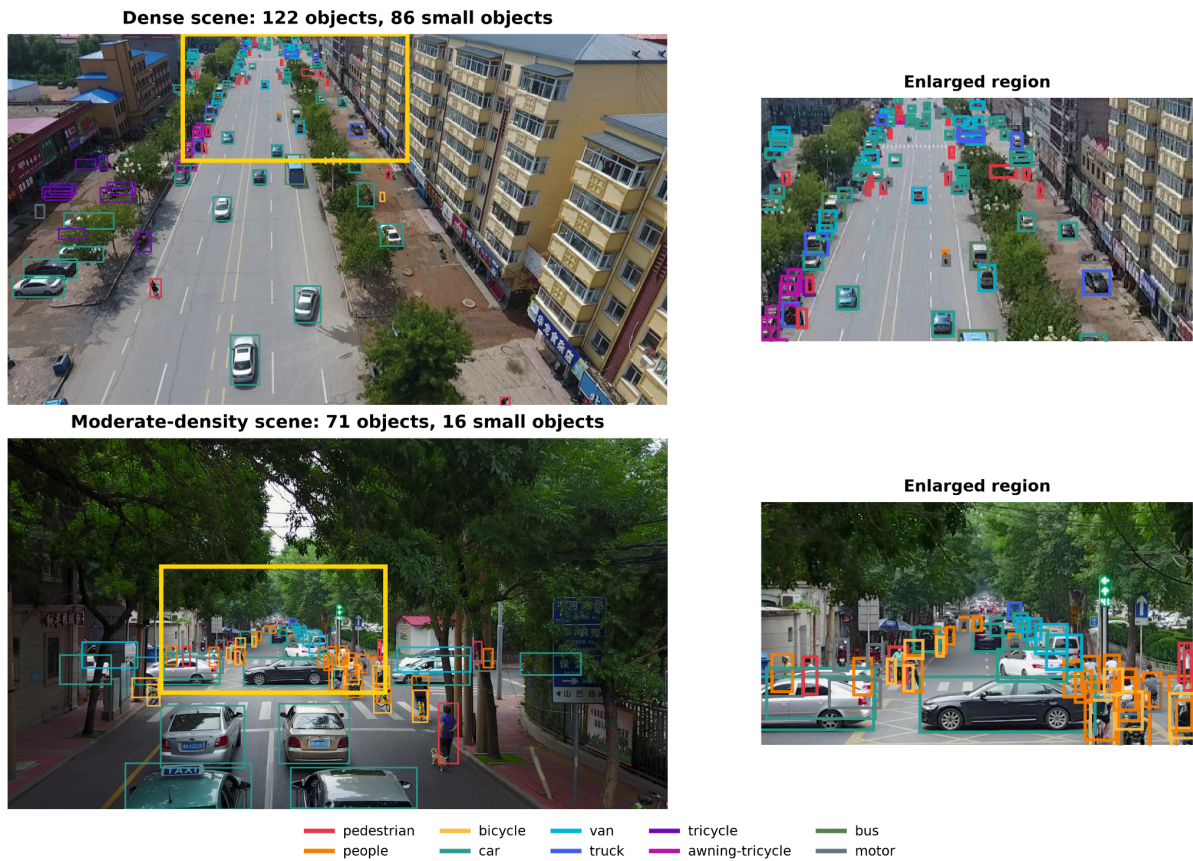


Figure 4. Representative validation images from the VisDrone2019-DET dataset.

As shown in **Figure 4**, the annotations are distributed non-uniformly and are often concentrated in a small number of local regions. The high-density scene contains 122 annotated objects, including 86 small objects, whereas the moderate-density scene contains 71 annotated objects, including 16 small objects. In the full-scene views, many pedestrians and vehicles occupy only a few pixels and are difficult to distinguish visually. The enlarged regions make the same instances substantially easier to observe, demonstrating how local cropping can preserve object detail that may otherwise be

lost when the entire image is resized to the detector input resolution. The difference between the two scenes also indicates that a fixed number of slices is unlikely to be equally efficient for every image.

After preprocessing and annotation conversion, the dataset used in this research consists of 8,629 images and 457,066 object labels. The dataset was divided into training, validation, and testing subsets. The training set was used to optimize the model parameters, the validation set was used to monitor model performance and support experimental adjustment, and the test set was used for final evaluation. This split provides a consistent experimental setting for comparing different inference strategies.

Overall, VisDrone2019-DET provides a suitable experimental foundation for this research. Its high-resolution UAV images, multi-class annotations, dense object distribution, and large number of small objects make it appropriate for evaluating the performance of object detection models in challenging aerial scenes.

3.1.2. Dataset Characteristics and Analysis

A key characteristic of VisDrone2019-DET is the dominance of small objects. Many objects in the dataset appear at very small scales relative to the full image size. This property makes the dataset particularly suitable for studying small object detection, where object details are easily lost during image resizing. In full-image inference, the entire high-resolution image is usually resized to a fixed input size before being passed into the detection model. As a result, small objects may become only a few pixels in size, reducing the amount of visual information available for feature extraction, localization, and classification.

To better understand the object scale distribution, the bounding boxes in the dataset were analyzed based on COCO-style object size categories, including small, medium, and large objects. The analysis shows that VisDrone2019-DET is strongly dominated by small objects, with 285,028 small object labels, accounting for approximately 62.36% of all object instances. This confirms that the dataset is highly suitable for the research problem, as most objects are vulnerable to information loss when the full image is resized before inference.

Figure 5 reveals three properties that are directly relevant to the proposed method. First, the class distribution is strongly imbalanced, with cars and pedestrians accounting for a large proportion of all annotations. Second, the relative-area distribution is concentrated near the small-object threshold. Using a relative-area threshold of 0.0004, 211,726 of the 457,065 valid annotations, corresponding to 46.32%, are categorized as small objects. Third, object density varies considerably across images. The dataset contains an average of 52.97 objects per image, with a median of 42, a 90th percentile of 104, and a maximum of 902. The proportion of small objects also differs across the training, validation, and test splits, with respective

values of 46.10%, 40.68%, and 50.27%. These findings indicate that detection difficulty is not uniform across images or dataset partitions, motivating a content-adaptive strategy that allocates additional computation only to regions that are likely to benefit from higher-resolution processing.

In addition to small object scale, VisDrone2019-DET also contains dense object distributions. Many images include a large number of objects within the same frame, with an average object density ranging from approximately 46 to 70 labels per image. Some images contain more than 900 objects, creating highly crowded scenes. Such high-density conditions introduce additional challenges, including overlapping bounding boxes, occlusion, and difficulty in separating nearby objects.

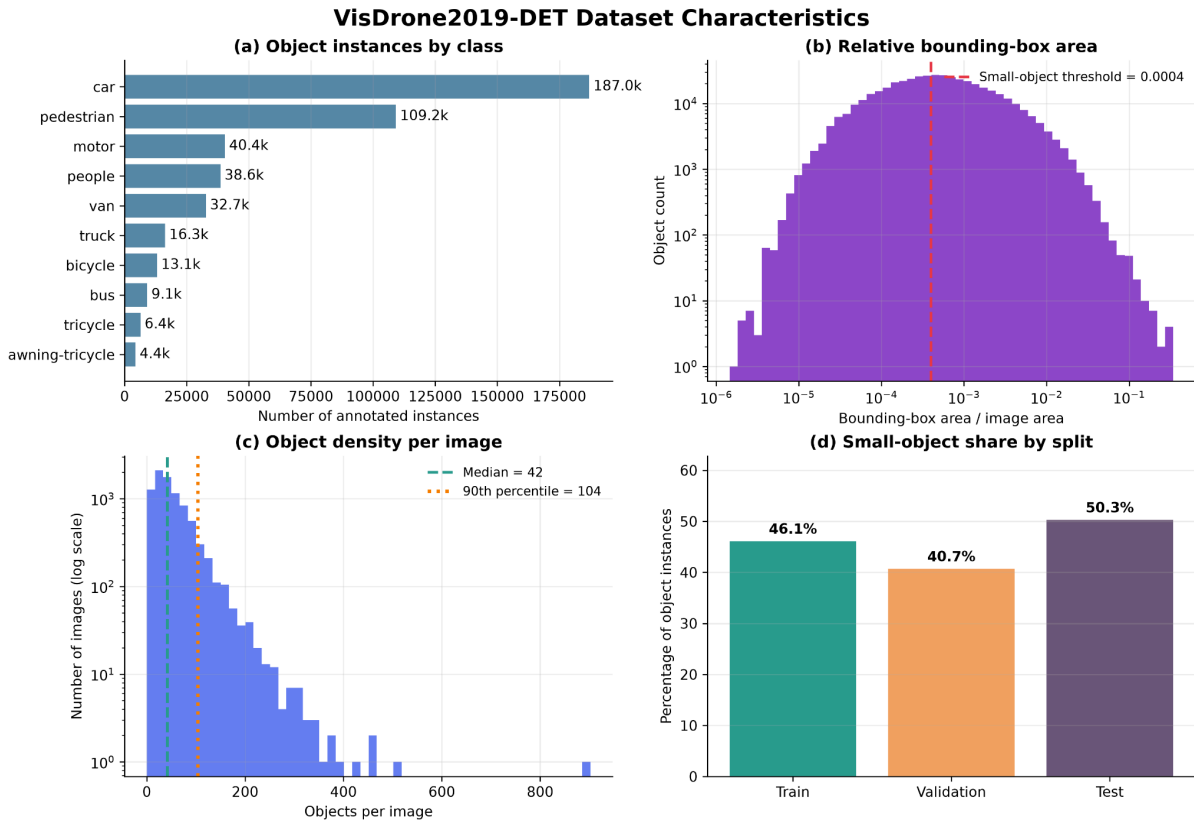


Figure 5. Object size distribution and object density distribution of the VisDrone2019-DET dataset.

The spatial distribution of objects also contributes to the difficulty of the dataset. Objects are not always evenly distributed across the entire image. In many cases, they are concentrated in specific regions such as roads, traffic intersections, and parking areas. Some objects also appear close to image boundaries or are partially cut off. These characteristics are important for this research because slicing-based inference methods depend on slice size, overlap ratio, and the position of objects across sliced regions.

To support a more detailed analysis, each image can be described using metadata such as high density, many small objects, high small-object ratio, border-heavy, and low density. These metadata groups help identify different levels of detection difficulty and support a more meaningful comparison between full-image inference

and slicing-based inference. For example, high-density images can be used to evaluate the model's ability to detect crowded small objects, while border-heavy images can help analyze the effect of slicing overlap and post-processing on objects located near slice boundaries.

The characteristics of VisDrone2019-DET directly support the motivation for using SAHI in this research. SAHI performs inference by dividing a high-resolution image into smaller slices instead of resizing the entire image at once. This allows small objects to retain a larger relative size within each slice, potentially improving feature extraction and increasing recall.



Figure 6. Comparison between full-image inference and slicing-based inference using SAHI.

Figure 6 shows that full-image inference processes the complete scene in a single detector pass, but all objects are simultaneously downscaled to fit the model input. In contrast, slicing divides the original image into overlapping regions and processes each region independently. When a slice is resized to the detector input resolution, the objects inside it occupy a larger proportion of the input and retain more visual detail. However, processing every slice also increases the number of detector forward passes, including for regions that contain mostly background. This trade-off between small-object visibility and computational cost provides the central motivation for learning where sliced inference should be applied.

In summary, the dataset analysis shows that VisDrone2019-DET is not only suitable for training and evaluating YOLO-based object detection models, but also appropriate for investigating the effectiveness of slicing-based inference. The dataset reflects the main challenges addressed in this research, including small object scale, high object density, occlusion, overlap, and complex UAV viewpoints. These characteristics make VisDrone2019-DET an appropriate benchmark for studying small object detection in high-resolution aerial images.

3.2. Experiment settings

All experiments were implemented in PyTorch and executed on a single NVIDIA Tesla T4 GPU with CUDA enabled. We used YOLO11s as the frozen base detector and trained only the RL slicing policy. The detector cache used to construct RL states was generated with image size 1280, confidence threshold 0.01, IoU threshold 0.7, and

maximum 3000 detections per image. All ten VisDrone2019-DET classes were retained, and experiments followed the official train/validation/test splits.

The RL policy was trained as a dueling Double DQN with AdamW, learning rate 1×10^{-4} , discount factor 0.95, batch size 512, replay buffer size 50,000, and 3-step returns. Training was performed for 20,000 episodes with 16 parallel environments. Prioritized replay, curriculum learning, soft target updates with $\tau = 0.005$, and cosine learning-rate scheduling were enabled. The detector was kept fixed during policy training, while cached detector features and hard-region annotations were used to construct the RL state and reward.

During inference, YOLO11s processes the full image at 1280 resolution and each selected ROI at 512 resolution. RL-SAHF uses at most four slice attempts and accepts at most four slices per image. ROI pre-filtering retains the top three candidates, and crop inference is performed in batches of three. A crop must pass the novelty-based acceptance gate before its detections are fused with full-image detections using class-aware Cluster-DIoU NMS with merge threshold 0.5. Normal inference uses output confidence threshold 0.30. Benchmark evaluation uses output confidence threshold 0.01, detector IoU threshold 0.70, and a maximum of 3000 detector outputs. Runtime measurements exclude the first ten warm-up images and disable cache usage. AP is computed class-wise using 101-point interpolated precision-recall curves at IoU thresholds from 0.50 to 0.95 with step 0.05. Precision, Recall, Recall-small, and false positives per image use the primary IoU threshold of 0.45. Evaluation retains at most 500 detections per class per image. Small objects are defined by relative bounding-box area not greater than 0.0004. Stratified sampling uses seed 42.

3.3. Results & Analysis

3.3.1. Evaluation metrics

In the context of small object detection within high-resolution images, a comprehensive evaluation must encompass both detection accuracy and inference efficiency. Specifically, detection accuracy metrics are utilized to quantify the model’s capability to accurately localize and identify objects, with a particular emphasis on small-scale instances. Efficiency metrics are used to measure the computational cost required by each inference strategy. Therefore, this thesis reports two main groups of metrics: **AP Metrics** and **Efficiency Metrics**.

AP Metrics. The first group of metrics evaluates the detection performance of the model. Since the proposed method aims to improve object detection in high-resolution drone images, especially for small objects, this thesis reports AP , AP_{50} , AP_{75} , Precision, Recall, and Recall-small. AP , AP_{50} and AP_{75} are used to measure detection quality under different IoU thresholds. Precision and Recall are used to analyze the balance between false positives and missed detections. Recall-small is additionally

reported because the thesis focuses on small object detection, where missing small objects is one of the main problems.

AP50 is the Average Precision computed at IoU threshold 0.5:

$$\frac{1}{N} \sum_{i=1}^N AP_i, IoU = 0.5 \quad (13)$$

AP is computed as the average precision over multiple IoU thresholds from 0.50 to 0.95:

$$AP_{50-95} = \frac{1}{10} \sum_{i=0.50, 0.55, \dots}^{0.95} AP_i \quad (14)$$

where AP_i denotes the Average Precision computed at IoU threshold (i).

AP75 is the Average Precision computed at IoU threshold 0.75:

$$AP_{75} = \frac{1}{N} \sum_{i=1}^N AP_i, IoU = 0.75 \quad (15)$$

Efficiency Metrics. The second group of metrics evaluates the inference efficiency of each method. This is necessary because slicing-based methods can improve small object detection but may introduce additional computational cost. Therefore, besides detection accuracy, this thesis reports Speed (images/second)

Speed measures the number of images processed per second:

$$Speed_{img/s} = \frac{N_{image}}{T_{agent} + T_{detector}} \quad (16)$$

Overall, AP Metrics and Efficiency Metrics provide a balanced evaluation of the proposed method. AP Metrics show whether the method improves detection quality, especially for small objects, while Efficiency Metrics show whether the method reduces the computational cost of processing high-resolution images.

3.3.2. Result Analysis

Table 2 compares the proposed RL-SAHl framework with conventional SAHI and ASAHl on the VisDrone2019-DET-test. RL-SAHl achieves 29.25 AP, 49.30 AP50, and 29.88 AP75. Although ASAHl obtains a slightly higher overall AP of 30.40, RL-SAHl produces the highest AP50 and AP75 among the compared methods. In particular, relative to SAHI, RL-SAHl improves AP, AP50, and AP75 by 4.72, 6.24, and 5.12 percentage points, respectively. Compared with ASAHl, it improves AP50 by 3.70 points and AP75 by 4.68 points. These results indicate that the learned region-selection policy can identify informative areas in which crop-level inference recovers detections that are weak or missed during full-image processing.

The scale-specific results further support this observation. RL-SAHl achieves the highest AP50 for small, medium, and large objects, with values of 40.74, 64.95, and 78.07, respectively. The gain for small objects is particularly important because

small-instance recovery is the principal objective of the proposed framework. RL-SAHl improves AP50 for small objects by 5.94 points over SAHI and 6.94 points over ASAHl, showing that selective high-resolution processing is more effective than uniformly processing all image regions.

In addition to detection quality, RL-SAHl provides the highest throughput, reaching 8.60 images/s compared with 5.04 images/s for SAHI and 4.88 images/s for ASAHl. This corresponds to speed increases of 70.63% and 76.23%, respectively. The result suggests that the RL policy reduces redundant slice inference by directing computation toward a small set of candidate regions. Nevertheless, the lower overall AP than ASAHl indicates that further improvement is needed to maintain localization quality consistently across the complete IoU range from 0.50 to 0.95.

Table 2. Results on VisDrone2019-DET-test. Best results in **bold**.

Method	Speed (img/s)	AP	AP ₅₀	AP ₇₅	AP _{50-s}	AP _{50-m}	AP _{50-l}
SAHI (512 px)	5.04	24.53	43.06	24.76	34.8	59.82	71.05
ASAHl	4.88	30.4	45.6	25.2	33.8	59.7	60.7
RL-SAHl	8.60	29.25	49.3	29.88	40.74	64.95	78.07

The component ablation study in **Table 3** demonstrates that the full RL-SAHl design benefits from the joint use of detection-aware state representation, history information, hard-region supervision, and cost/overlap regularization. The complete model achieves 33.16 AP, 56.31 AP50, and 32.83 AP75. Removing any individual component reduces AP by 1.19–1.30 points and AP50 by 4.51–4.65 points. The detection map and history map are particularly important because they provide complementary spatial information: the detection map highlights uncertain and small-object proposals, whereas the history map discourages the agent from repeatedly selecting previously explored regions.

Table 3. Component ablation on VisDrone2019-DET-val.

Detection map	History map	Hard-region reward	Cost overlap	AP	AP50	AP75
	✓	✓	✓	31.86	51.66	32.73
✓		✓	✓	31.86	51.74	32.69
✓	✓		✓	31.89	51.76	32.69
✓	✓	✓		31.97	51.80	32.70
✓	✓	✓	✓	33.16	56.31	32.83

Removing the hard-region reward decreases AP from 33.16 to 31.89, confirming the value of training supervision derived from missed small-object regions. Likewise, removing the cost/overlap term reduces AP to 31.97, indicating that explicit penalties for oversized or redundant ROIs help the policy allocate its limited crop budget more

effectively. In contrast, AP75 changes by less than 0.14 points across the ablated variants. Since the base detector is fixed and the ablated components primarily affect region selection rather than bounding-box regression, this pattern suggests that these components mainly improve the probability of recovering relevant objects, particularly at the AP50 operating point, rather than substantially changing the geometric precision of individual detections.

Table 4 positions RL-SAHl within the broader VisDrone2019-DET literature. RL-SAHl achieves 33.16 AP, 56.31 AP50, and 32.83 AP75 while processing 8.50 images/s. Its overall AP is higher than that of Faster R-CNN, ZoomDet, SAHl, AdaZoom, and ClusDet in the reported comparison. Although its AP remains below QueryDet, AD-Det, and ASAHl, RL-SAHl provides a notably stronger accuracy-throughput trade-off. For example, it is 3.09 times faster than QueryDet, 4.36 times faster than AD-Det, and 1.62 times faster than ASAHl.

The AP75 result is also competitive. RL-SAHl exceeds ASAHl by 4.63 points and QueryDet by 13.33 points at IoU 0.75, suggesting that the proposed framework can preserve useful localization quality while limiting the number of high-resolution crop evaluations. Therefore, RL-SAHl should not be interpreted as the method with the highest absolute AP in every setting; rather, its main contribution is an adaptive inference strategy that achieves competitive detection performance with substantially higher throughput.

Because the methods in **Table 4** use different detectors, backbones, training procedures, input resolutions, and hardware platforms, these values should be described as a contextual comparison rather than a strictly controlled head-to-head benchmark.

Table 4. State-of-the-art comparison on VisDrone2019-DET-val.

Method	AP	AP50	AP75	Speed (img/s)
Faster R-CNN [30]	12.3	24.9	9.6	2.57
ZoomDet [4]	22.8	39.9	18.9	1.29
SAHl [10]	28.7	42.2	24.1	3.69
AdaZoom [2]	31.22	56.16	31.22	1.52
ClusDet [11]	32.4	56.2	18.9	1.29
QueryDet [12]	33.9	56.1	19.5	2.75
AD-Det [14]	35.9	58.8	37.2	1.95
ASAHl [10]	36.0	56.8	28.2	5.26
RL-SAHl	33.16	56.31	32.83	8.50

3.3.3. Qualitative Examples

Qualitative analysis was conducted to complement the aggregate evaluation metrics and examine the behaviour of RL-SAHl under different scene conditions. Four

illustrative images were selected from the annotated evaluation split to show four commonly observed behaviour patterns: a small-object recovery case, a crowded-scene trade-off, a limited-change case, and a failure case. These examples are intended to illustrate model behaviour rather than provide a statistical evaluation.

Unless otherwise specified, true positives and false positives are determined through class-consistent one-to-one matching at an IoU threshold of 0.5. Small objects are defined as , consistent with the quantitative evaluation protocol.

Figure 7 contains twelve panels arranged in three rows and four columns. Each column represents one scene, while the rows show ground-truth annotations, YOLO full-image predictions, and final RL-SAHI predictions. Green boxes represent ground-truth objects, orange boxes represent YOLO full-image predictions, cyan boxes represent final RL-SAHI detections retained from the full-image pass, and magenta boxes represent final detections introduced by accepted slices.



Figure 7. Qualitative comparison between YOLO full-image inference and RL-SAHI on four illustrative VisDrone2019-DET images.

Panels (a, e, i) show a successful small-object recovery case. YOLO full-image inference matches 38 of the 71 ground-truth objects, including 9 of the 30 small objects. RL-SAHI increases the number of matched objects to 42 and the number of matched small objects to 12. At the same time, false positives decrease from 23 to 13. The magenta boxes in panel (i) represent predictions introduced by accepted slices, demonstrating that local high-resolution inference can recover objects whose visual

features are weakened when the complete image is resized to the detector input resolution.

Panels (b, f, j) show a crowded scene containing 102 ground-truth objects. Both methods match 83 objects. RL-SAHl increases the number of matched small objects from 15 to 16 out of 25, but false positives increase from 27 to 35. This result illustrates a recall–precision trade-off: additional crop inference recovers one small object but also introduces eight additional false-positive detections. In dense regions, closely located and partially overlapping objects may increase ambiguity during crop-level inference and post-processing.

Panels (c, g, k) present a limited-change case. YOLO full-image inference matches 62 objects, while RL-SAHl matches 60. Both methods match 28 of the 36 small objects, and the number of false positives decreases from 11 to 9. Therefore, sliced inference provides no improvement in small-object recall for this image. The reduction in total true positives suggests that some reliable full-image detections may have been affected during crop fusion, confidence filtering, or non-maximum suppression.

Panels (d, h, l) show a failure case. YOLO full-image inference matches 52 of the 55 ground-truth objects, including 11 of the 13 small objects. RL-SAHl matches 45 objects, including 7 small objects, while false positives increase from 57 to 66. Because the RL-SAHl output is produced by merging full-image and crop detections, the reduction in true positives cannot be explained solely by the absence of a useful crop. It may indicate a failure in crop acceptance or detection fusion. Possible causes include crop predictions suppressing reliable full-image detections during non-maximum suppression, class conflicts, or localization changes after fusion. This case highlights the need for better policy calibration, crop-quality estimation, and fusion rules that preserve reliable baseline detections.

Overall, **Figure 7** shows that RL-guided slicing can recover additional small objects when the selected regions provide sufficient scale gain and useful visual information. However, its effectiveness varies across images. Additional crop inference may provide only limited improvement in crowded scenes, offer no benefit when the full-image detector already performs well, or reduce performance when unreliable crop detections interfere with correct baseline predictions. These examples provide qualitative context for the aggregate quantitative results and highlight several mechanisms that may influence the overall accuracy–efficiency trade-off.

4. Conclusion & Future Work

4.1. Conclusion

This thesis presented RL-SAHl, a slicing framework in which a Dueling Double DQN policy decides where a high-resolution detector should be re-applied, instead of tiling the image with a fixed grid. The detector is executed once per image and is never

modified; the policy observes only the evidence that this single pass already produces - a semantic feature vector, an objectness heat map, compressed multi-level spatial maps, detection statistics and the history of regions already attempted -and navigates a region of interest through eleven discrete actions until it emits a terminal action. A lightweight pre-filter then ranks the proposed regions on the same cached evidence and forwards only the highest-scoring ones, so the detector cost of an image is bounded at one full-image pass plus a fixed number of crops, known before any region is proposed. Because the policy is trained against a terminal crop-outcome reward that measures the net new detections a candidate crop contributes after class-aware non-maximum suppression, training optimises exactly the criterion that decides acceptance at inference.

On VisDrone2019-DET, RL-SAHl reaches 28.38 AP, 47.50 AP50, and 29.15 AP75 on the test-dev split, and 31.43 AP, 53.31 AP50, and 31.36 AP75 on the validation split. These results show that the proposed policy can identify image regions for which additional high-resolution inference is useful, while avoiding uniform slicing over the entire image. The contribution of RL-SAHl is therefore not an absolute claim of state-of-the-art accuracy, but an adaptive slicing mechanism that uses detector-derived evidence to allocate crop evaluations to informative regions. Quantitative comparisons with the SAHl (512 px) baseline should be interpreted only under the same split and evaluation protocol, whereas results reported for ASAHl serve as contextual reference because they were obtained with a different detector and hardware. The margin at the strict threshold is the substantive result: regions chosen from detector evidence are localised more precisely than regions produced by tiling, which is the property a content-conditioned slicing policy is expected to buy.

The contribution is therefore not a new state of the art in absolute accuracy -larger two-stage detectors on heavier backbones remain ahead -but a demonstration that a learned slicing policy reaches a markedly better accuracy-per-crop operating point than a fixed grid, while requiring no additional trainable module and no modification to the detector. The evaluation protocol and the component ablation reported alongside it make the sources of that behaviour explicit rather than assumed.

4.2. Future Works

Three directions follow directly from the limitations of this study. The first concerns the data. The training and validation partitions share 24 sequence prefixes, so consecutive frames of the same flight appear on both sides of the split, and every validation figure reported here should be read as preliminary until the split is rebuilt by sequence and the policy retrained. The component ablation inherits the same caveat, since it was obtained by disabling components at inference on a single trained policy rather than by retraining each variant; repeating it as a retraining study on a leak-free split is the first step.

The second concerns how regions are proposed. The proposal stage is currently sequential: each rollout assembles its own state and runs to termination before the next begins, so the policy overhead grows with the number of attempts even though the underlying detector evidence is shared across all of them. Extrapolating the per-crop cost from the fixed-grid measurements, the three crops that RL-SAHF issues account for roughly 90 - 120 ms it adds to each image, which places about three quarters of that cost in the proposal stage rather than in detection. Future work replaces the sequential rollout with a parallel formulation in which all candidate regions advance together over a shared state; removing that overhead would bring the per-image cost close to the detector work the crops actually require, and would make the reduction in effective GFLOPs visible in wall-clock terms. The same redesign is what makes a dynamic budget practical: once rollouts are evaluated jointly, a learned image-level stop criterion can end the proposal loop early, so that a sparse scene no longer pays the same detector cost as a dense one.

The third concerns generality. The detector is frozen throughout, and a single architecture was evaluated on a single benchmark. Fine-tuning the detector on the crop distribution that the policy itself produces, and validating the method on a second detector family and a second high-resolution dataset, would establish that the observed precision-per-crop advantage is a property of the approach rather than of one configuration.

References

1. Gao, M., Yu, R., Li, A., Morariu, V.I., Davis, L.S.: Dynamic Zoom-in Network for Fast Object Detection in Large Images, <https://arxiv.org/abs/1711.05187>.
2. Najibi, M., Singh, B., Davis, L.S.: AutoFocus: Efficient Multi-Scale Inference, <https://arxiv.org/abs/1812.01600>.
3. Xu, J., Li, Y., Wang, S.: AdaZoom: Adaptive Zoom Network for Multi-Scale Object Detection in Large Scenes, <https://arxiv.org/abs/2106.10409>.
4. Wang, T., Lin, C., Tang, C., Zhou, J., Xiong, D., Li, J., Zhao, J., Lv, J.: Adaptive Image Zoom-in with Bounding Box Transformation for UAV Object Detection, <https://arxiv.org/abs/2602.07512>.
5. Caicedo, J.C., Lazebnik, S.: Active Object Localization with Deep Reinforcement Learning, <https://arxiv.org/abs/1511.06015>.
6. Uzkont, B., Yeh, C., Ermon, S.: Efficient Object Detection in Large Images using Deep Reinforcement Learning, <https://arxiv.org/abs/1912.03966>.
7. Zhang, J., Yang, X., He, W., Ren, J., Zhang, Q., Zhao, T., Bai, R., He, X., Liu, J.: Scale Optimization Using Evolutionary Reinforcement Learning for Object Detection on Drone Imagery, <https://arxiv.org/abs/2312.15219>.
8. Wang, D., Sapkota, H., Yu, Q.: Adaptive Important Region Selection with Reinforced Hierarchical Search for Dense Object Detection, <https://neurips.cc/virtual/2024/poster/94227>.
9. Akyon, F.C., Altinuc, S.O., Temizel, A.: Slicing Aided Hyper Inference and Fine-tuning for Small Object Detection, <https://arxiv.org/abs/2202.06934>.
10. Moretti, F., Jin, Y., Mario, G.: Adaptive Slicing-Assisted Hyper Inference for Enhanced Small Object Detection in High-Resolution Imagery, <https://arxiv.org/abs/2604.19233>.
11. Yang, F., Fan, H., Chu, P., Blasch, E., Ling, H.: Clustered Object Detection in Aerial Images, <https://doi.org/10.1109/ICCV.2019.00840>.
12. Yang, C., Huang, Z., Wang, N.: QueryDet: Cascaded Sparse Query for Accelerating High-Resolution Small Object Detection, <https://arxiv.org/abs/2103.09136>.
13. Liu, K., Fu, Z., Jin, S., Chen, Z., Zhou, F., Jiang, R., Chen, Y., Ye, J.: ESOD: Efficient Small Object Detection on High-Resolution Images, <https://arxiv.org/abs/2407.16424>.
14. Li, Z., Lian, S., Pan, D., Wang, Y., Liu, W.: AD-Det: Boosting Object Detection in UAV Images with Focused Small Objects and Balanced Tail Classes. (2025). <https://doi.org/10.48550/ARXIV.2504.05601>.
15. Siddique, A., Zhengzhou, L., Azeem, A., Yuting, Z., Li, Y.: Dynamic Adaptive Region Transformer for Tiny-Object Detection in Remote Sensing. <https://doi.org/10.1109/TGRS.2025.3582173>.

16. Khanam, R., Hussain, M.: YOLOv11: An Overview of the Key Architectural Enhancements, <https://arxiv.org/abs/2410.17725>.
17. Feng, Q., Xu, X., Wang, Z., Feng, Q., Xu, X., Wang, Z.: Deep learning-based small object detection: A survey, <https://doi.org/10.3934/mbe.2023282>.
18. Aldubaikhi, A., Patel, S.: Advancements in Small-Object Detection (2023–2025): Approaches, Datasets, Benchmarks, Applications, and Practical Guidance, <https://doi.org/10.3390/app152211882>.
19. Zhu, P., Wen, L., Bian, X., Ling, H., Hu, Q.: Vision Meets Drones: A Challenge, <https://arxiv.org/abs/1804.07437>.
20. Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G., et al.: Human-level control through deep reinforcement learning. *Nature* 518(7540), 529–533 (2015), <https://doi.org/10.1038/nature14236>.
21. van Hasselt, H., Guez, A., Silver, D.: Deep Reinforcement Learning with Double Q-learning. In: AAI (2016), <https://arxiv.org/abs/1509.06461>.
22. Wang, Z., Schaul, T., Hessel, M., van Hasselt, H., Lanctot, M., de Freitas, N.: Dueling Network Architectures for Deep Reinforcement Learning. In: ICML (2016), <https://arxiv.org/abs/1511.06581>.
23. Schaul, T., Quan, J., Antonoglou, I., Silver, D.: Prioritized Experience Replay. In: ICLR (2016). <https://arxiv.org/abs/1511.05952>.
24. Hessel, M., Modayil, J., van Hasselt, H., Schaul, T., Ostrovski, G., Dabney, W., Horgan, D., Piot, B., Azar, M., Silver, D.: Rainbow: Combining Improvements in Deep Reinforcement Learning. In: AAI (2018). <https://arxiv.org/abs/1710.02298>.
25. Lillicrap, T.P., Hunt, J.J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., Wierstra, D.: Continuous control with deep reinforcement learning. In: ICLR (2016), <https://arxiv.org/abs/1509.02971>.
26. Loshchilov, I., Hutter, F.: Decoupled Weight Decay Regularization. In: ICLR (2019), <https://arxiv.org/abs/1711.05101>.
27. Loshchilov, I., Hutter, F.: SGDR: Stochastic Gradient Descent with Warm Restarts. In: ICLR (2017), <https://arxiv.org/abs/1608.03983>.
28. Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction, 2nd edn. MIT Press, Cambridge, MA (2018).
29. Lin, T.-Y., Maire, M., Belongie, S., Bourdev, L., Girshick, R., Hays, J., Perona, P., Ramanan, D., Zitnick, C.L., Dollár, P.: Microsoft COCO: Common Objects in Context, <https://arxiv.org/abs/1405.0312>.
30. Wang, X., Shrivastava, A., Gupta, A.: A-Fast-RCNN: Hard Positive Generation via Adversary for Object Detection, <https://arxiv.org/abs/1704.03414>.